



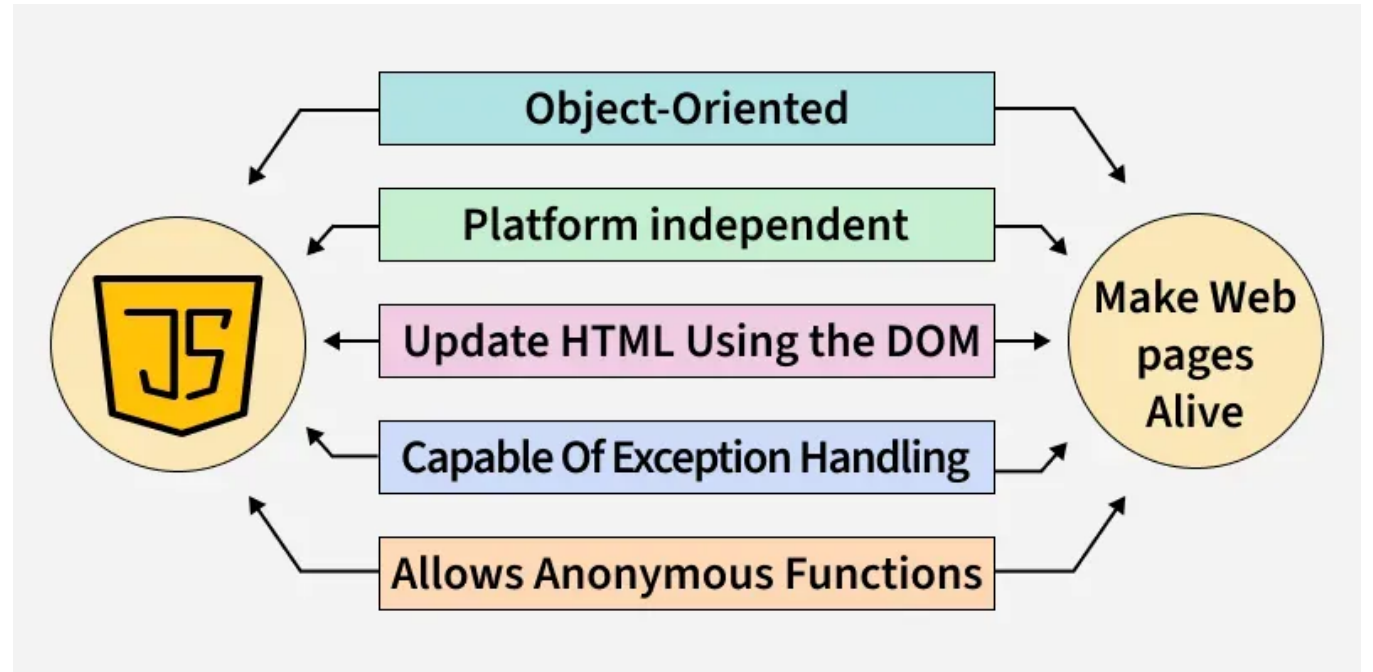
FULL-STACK APPLICATION DESIGN AND DEVELOPMENT

COURSE 2 - JAVASCRIPT

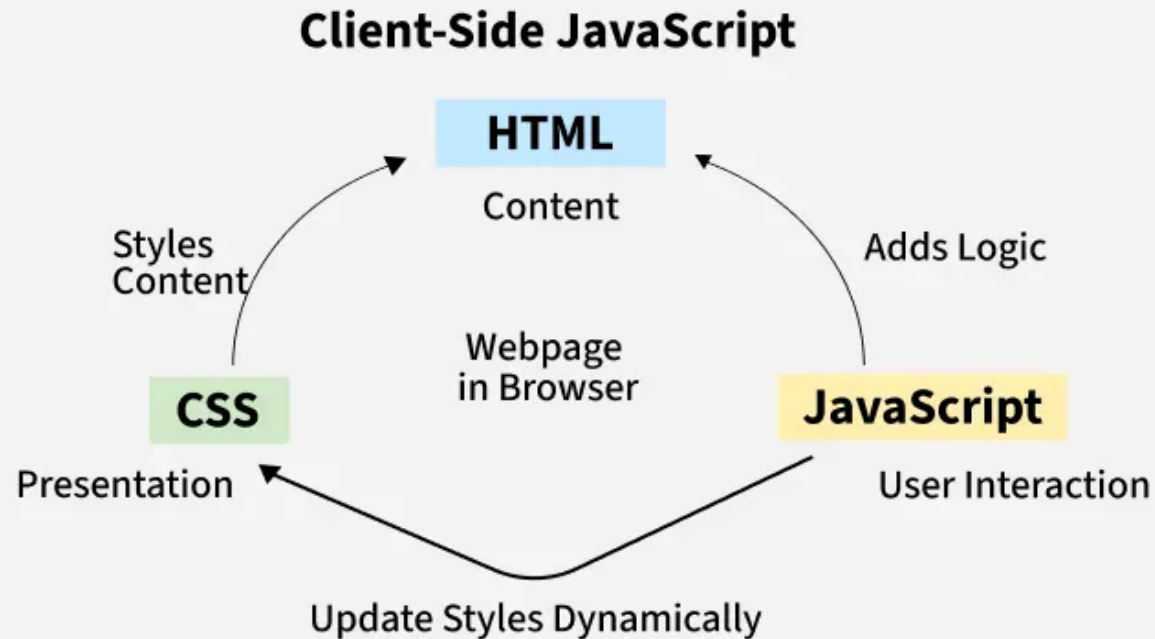


WHAT IS JAVASCRIPT?

- JavaScript is the programming language of the web.
- It can calculate, manipulate and validate data.
- It can update and change both HTML and CSS.
- JavaScript is one of the **3 languages** all web developers must learn: **HTML** to define the content of web pages, **CSS** to specify the layout of web pages and **JavaScript** to program the behavior of web pages.



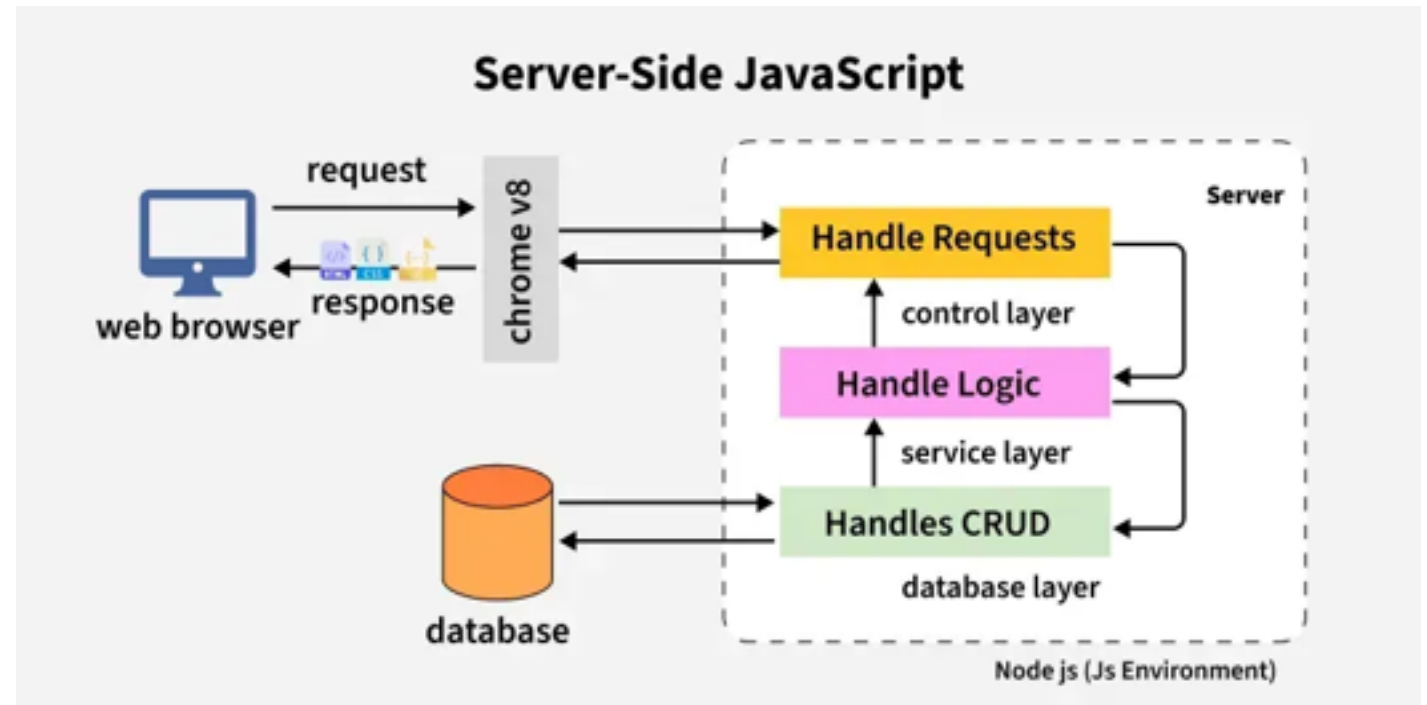
CLIENT SIDE



- Involves controlling the browser and its DOM (Document Object Model).
- Handles user events like clicks and form inputs.
- Common libraries include AngularJS, ReactJS, and VueJS.

SERVER SIDE

- Involves interacting with databases, manipulating files, and generating responses.
- Node.js and frameworks like Express.js are widely used for server-side JavaScript, enabling full-stack development.



JAVASCRIPT FUNCTIONS

- JavaScript functions are **defined** with the `function` keyword.
- You can use a function **declaration** or a function **expression**.
- Functions are **declared** with the following syntax:

```
function myFunction(a, b) {  
    return a * b;  
}
```
- Declared functions are not executed immediately. They are "saved for later use", and will be executed later, when they are invoked (called upon).

JAVASCRIPT FUNCTIONS

- JavaScript functions can be used as values:

```
function myFunction(a, b) {  
    return a * b;  
}  
let x = myFunction(4, 3);
```

- They can also be used in expressions:

```
function myFunction(a, b) {  
    return a * b;  
}  
let x = myFunction(4, 3) * 2;
```

JAVASCRIPT FUNCTIONS

- JavaScript functions have both **properties** and **methods**.
- The arguments.length property returns the number of arguments received when the function was invoked:

```
function myFunction(a, b) {  
    return arguments.length;  
}
```

- The toString() method returns the function source code as a string:

```
function myFunction(a, b) {  
    return a * b;  
}  
let text = myFunction.toString();
```

JAVASCRIPT FUNCTIONS

- Function **parameters** are the **names** listed in the function definition.
- Function **arguments** are the real **values** passed to (and received by) the function.

```
function functionName(parameter1, parameter2, parameter3) {  
    // code to be executed  
}
```
- JavaScript function definitions do not specify data types for parameters.
- JavaScript functions do not perform type checking on the passed arguments and do not check the number of arguments received.

JAVASCRIPT FUNCTIONS

- The rest parameter (...) allows a function to treat an indefinite number of arguments as an array:

```
function sum(...args) {  
    let sum = 0;  
    for (let arg of args) sum += arg;  
    return sum;  
}
```

```
let x = sum(4, 9, 16, 25, 29, 100, 66, 77);
```

JAVASCRIPT FUNCTIONS

Arguments are Passed by Value

- The parameters, in a function call, are the function's arguments.
- JavaScript arguments are passed by **value**.
- If a function changes an argument's value, it does not change the parameter's original value.
- **Changes to arguments are not visible (reflected) outside the function.**

JAVASCRIPT FUNCTIONS

- In JavaScript you can define functions as object methods.
- The following example creates an object (**myObject**), with two properties (**firstName** and **lastName**), and a method (**fullName**):

```
const myObject = {  
  firstName: "John",  
  lastName: "Doe",  
  fullName: function () {  
    return this.firstName + " " + this.lastName;  
  }  
}  
myObject.fullName();           // Will return "John Doe"
```

JAVASCRIPT FUNCTIONS

- If a function invocation is preceded with the `new` keyword, it is a constructor invocation.
- It looks like you create a new function, but since JavaScript functions are objects you actually create a new object:

```
// This is a function constructor
function myFunction(arg1, arg2) {
  this.firstName = arg1;
  this.lastName  = arg2;
}
// This creates a new object
const myObj = new myFunction("John", "Doe");
// This will return "John"
myObj.firstName;
```

- A constructor invocation creates a new object. The new object inherits the properties and methods from its constructor.

JAVASCRIPT FUNCTIONS

- The `call()` method is a predefined JavaScript method.
- It can be used to invoke (call) a method with an object as an argument.
- This example calls the **fullName** method of person, using it on **person1**:

```
const person = {
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
}
const person1 = {
  firstName: "John",
  lastName: "Doe"
}
const person2 = {
  firstName: "Mary",
  lastName: "Doe"
}
// This will return "John Doe":
person.fullName.call(person1);
```

JAVASCRIPT FUNCTIONS

- This example calls the **fullName** method of person, using it on **person2**:

```
const person = {
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
}
const person1 = {
  firstName: "John",
  lastName: "Doe"
}
const person2 = {
  firstName: "Mary",
  lastName: "Doe"
}
// This will return "Mary Doe"
person.fullName.call(person2);
```

JAVASCRIPT FUNCTIONS

- The `call()` method can accept arguments:

```
const person = {
  fullName: function(city, country) {
    return this.firstName + " " + this.lastName + ", " + city + ", " + country;
  }
}
const person1 = {
  firstName: "John",
  lastName: "Doe"
}
person.fullName.call(person1, "Oslo", "Norway");
```

JAVASCRIPT OBJECTS

- **Some methods for defining JavaScript objects:**
 - Using an Object Literal
 - Using `Object.create()`
 - Using `Object.assign()`

JAVASCRIPT OBJECTS

- An object literal is a list of property **key:values** inside curly braces `{ }`.
- `{firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"};`

```
// Create an Object
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 50,
  eyeColor: "blue"
};
```

JAVASCRIPT OBJECTS

- The `Object.create()` method creates an object from an existing object.

```
// Create an Object:
const person = {
  firstName: "John",
  lastName: "Doe"
};
// Create new Object
const man = Object.create(person);
man.firstName = "Peter";
```

JAVASCRIPT OBJECTS

- The `Object.assign()` method copies properties from one or more source objects to a target object.

```
// Create Target Object
const person1 = {
  firstName: "John",
  lastName: "Doe",
  age: 50,
  eyeColor: "blue"
};
// Create Source Object
const person2 = {firstName: "Anne", lastName: "Smith"};
// Assign Source to Target
Object.assign(person1, person2);
```

JAVASCRIPT OBJECTS

- The syntax for accessing the property of an object is:

```
// objectName.property  
let age = person.age;
```

- Or

```
//objectName["property"]  
let age = person["age"];
```

- You can add new properties to an existing object by simply giving it a value:

```
person.nationality = "English";
```

JAVASCRIPT OBJECTS

- A **value** given to a property will be a **default value** for all objects created by the constructor:

```
function Person(first, last, age, eyecolor) {  
  this.firstName = first;  
  this.lastName = last;  
  this.age = age;  
  this.eyeColor = eyecolor;  
  this.nationality = "English";  
}
```

JAVASCRIPT OBJECTS

- The delete keyword deletes a property from an object:

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 50,  
  eyeColor: "blue"  
};  
delete person.age;
```

- Or delete person["age"];
delete person["age"];

JAVASCRIPT OBJECTS

- Property values in an object can be other objects:

```
myObj = {  
  name: "John",  
  age: 30,  
  myCars: {  
    car1: "Ford",  
    car2: "BMW",  
    car3: "Fiat"  
  }  
}
```

- You can access nested objects using the dot notation or the bracket notation:

```
myObj.myCars.car2; or myObj.myCars["car2"]; or myObj["myCars"]["car2"];
```

JAVASCRIPT OBJECTS

- **Object methods** are actions that can be performed on objects.
- A method is a **function definition** stored as a **property value**.
- In the example below, this refers to the **person object**:
 - **this.firstName** means the **firstName** property of **person**.
 - **this.lastName** means the **lastName** property of **person**.

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  id: 5566,  
  fullName: function() {  
    return this.firstName + " " + this.lastName;  
  }  
};
```


JAVASCRIPT OBJECTS

- You access an object method with the following syntax:

objectName.methodName()

- If you invoke the **fullName property** with (), it will execute as a **function**:

```
name = person.fullName();
```

- If you access the **fullName property** without (), it will return the **function definition**:

```
name = person.fullName;
```

JAVASCRIPT OBJECTS

- Some solutions to display JavaScript objects are:
- Displaying the Object Properties in a Loop

```
// Create an Object
const person = {
  name: "John",
  age: 30,
  city: "New York"
};
// Build a Text
let text = "";
for (let x in person) {
  text += person[x] + " ";
};
```

JAVASCRIPT OBJECTS

- o Displaying the Object using Object.values()

```
// Create an Object
const person = {
  name: "John",
  age: 30,
  city: "New York"
};
// Create an Array
const myArray = Object.values(person);
// Stringify the Array
let text = myArray.toString();
```

JAVASCRIPT OBJECTS

- Displaying the Object using JSON.stringify()
- JavaScript objects can be converted to a string with JSON method JSON.stringify().
- JSON.stringify() is included in JavaScript and supported in all browsers.

```
// Create an Object
const person = {
  name: "John",
  age: 30,
  city: "New York"
};
// Stringify Object
let text = JSON.stringify(person);
```

JAVASCRIPT CLASSES

- Use the keyword `class` to create a class.
- Always add a method named `constructor()`:

```
class Car {  
  constructor(name, year) {  
    this.name = name;  
    this.year = year;  
  }  
}
```

- When you have a class, you can use the class to create objects:

```
const myCar1 = new Car("Ford", 2014);  
const myCar2 = new Car("Audi", 2019);
```

JAVASCRIPT CLASSES

- The constructor method is a special method:
 - It has to have the exact name "constructor";
 - It is executed automatically when a new object is created;
 - It is used to initialize object properties.
- If you do not define a constructor method, JavaScript will add an empty constructor method.

JAVASCRIPT CLASSES

- Class methods are created with the same syntax as object methods.
- Add any number of methods to the class:

```
class Car {  
  constructor(name, year) {  
    this.name = name;  
    this.year = year;  
  }  
  age() {  
    const date = new Date();  
    return date.getFullYear() - this.year;  
  }  
}  
const myCar = new Car("Ford", 2014);  
document.getElementById("demo").innerHTML = "My car is " + myCar.age() + " years old.";
```

JAVASCRIPT CLASSES

- To create a class inheritance, use the extends keyword.
- A class created with a class inheritance inherits all the methods from another class:

```
class Car {  
  constructor(brand) {  
    this.carname = brand;  
  }  
  present() {  
    return 'I have a ' + this.carname;  
  }  
}  
class Model extends Car {  
  constructor(brand, mod) {  
    super(brand);  
    this.model = mod;  
  }  
  show() {  
    return this.present() + ', it is a ' + this.model;  
  }  
}  
let myCar = new Model("Ford", "Mustang");  
document.getElementById("demo").innerHTML = myCar.show();
```


JAVASCRIPT CLASSES

- Classes also allow you to use getters and setters.
- It can be smart to use getters and setters for your properties, especially if you want to do something special with the value before returning them, or before you set them.
- To add getters and setters in the class, use the `get` and `set` keywords.

```
class Car {  
  constructor(brand) {  
    this.carname = brand;  
  }  
  get cnam() {  
    return this.carname;  
  }  
  set cnam(x) {  
    this.carname = x;  
  }  
}  
const myCar = new Car("Ford");  
document.getElementById("demo").innerHTML = myCar.cnam;
```

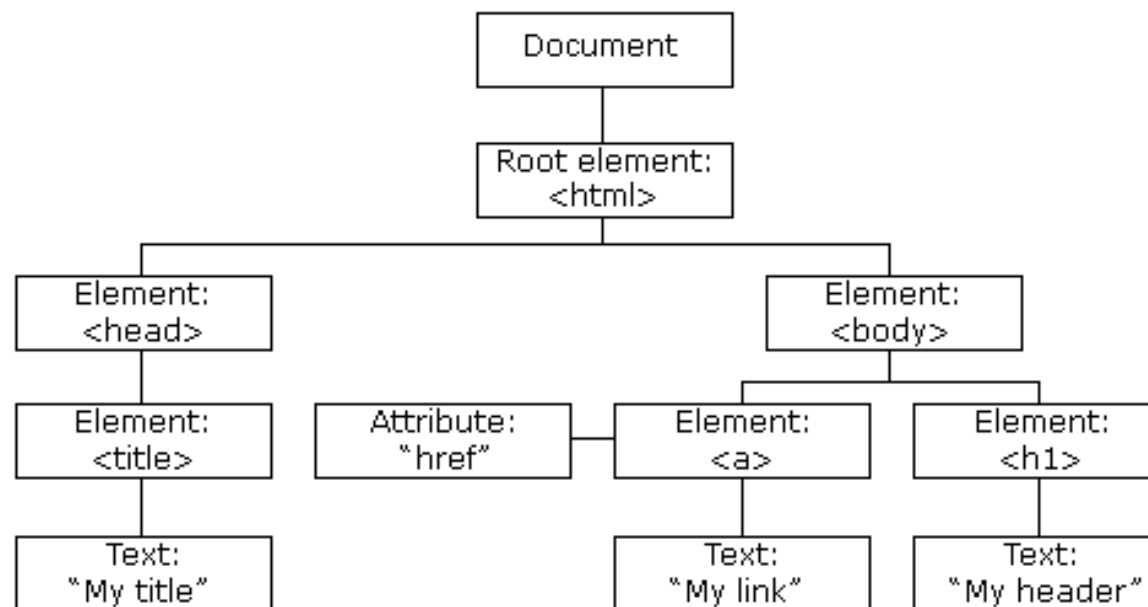
JAVASCRIPT CLASSES

- To use a *setter*, use the same syntax as when you set a property value, without parentheses:

```
class Car {  
  constructor(brand) {  
    this._carname = brand;  
  }  
  get carname() {  
    return this._carname;  
  }  
  set carname(x) {  
    this._carname = x;  
  }  
}  
const myCar = new Car("Ford");  
myCar.carname = "Volvo";  
document.getElementById("demo").innerHTML = myCar.carname;
```

JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

- When a web page is loaded, the browser creates a **Document Object Model** of the page.
- The **HTML DOM** model is constructed as a tree of **Objects**:



JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

- **What is the DOM?**
- The DOM is a W3C (World Wide Web Consortium) standard.
- The DOM defines a standard for accessing documents:
- "The W3C Document Object Model (DOM) is a platform and language-neutral interface that allows programs and scripts to dynamically access and update the content, structure, and style of a document."
- The W3C DOM standard is separated into 3 different parts:
 - Core DOM - standard model for all document types;
 - XML DOM - standard model for XML documents;
 - HTML DOM - standard model for HTML documents.

JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

- **What is the HTML DOM?**
- The HTML DOM is a standard **object** model and **programming interface** for HTML. It defines:
 - The HTML elements as **objects**;
 - The **properties** of all HTML elements;
 - The **methods** to access all HTML elements;
 - The **events** for all HTML elements.
- In other words: **The HTML DOM is a standard for how to get, change, add, or delete HTML elements.**

JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

- The HTML DOM can be accessed with JavaScript (and with other programming languages).
- In the DOM, all HTML elements are defined as **objects**.
- The programming interface is the properties and methods of each object.
- A **property** is a value that you can get or set (like changing the content of an HTML element).
- A **method** is an action you can do (like adding or deleting an HTML element).

```
<html><body>
```

```
<p id="demo"></p>
```

```
<script>
```

```
document.getElementById("demo").innerHTML = "Hello World!";
```

```
</script>
```

```
</body></html>
```

JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

- The most common way to access an HTML element is to use the `id` of the element.
- In the previous example the `getElementById` method used `id="demo"` to find the element.
- There are several ways to do this:
 - Finding HTML elements by `id`
 - Finding HTML elements by tag name
 - Finding HTML elements by class name
 - Finding HTML elements by CSS selectors
 - Finding HTML elements by HTML object collections

JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

Finding HTML elements by id:

- The easiest way to find an HTML element in the DOM, is by using the element id.
- This example finds the element with `id="intro"`:

```
const element = document.getElementById("intro");
```

- If the element is found, the method will return the element as an object.
- If the element is not found, element will contain `null`.

JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

Finding HTML elements by tag name:

- This example finds all <p> elements:

```
const element = document.getElementsByTagName("p");
```

- This example finds the element with `id="main"`, and then finds all <p> elements inside "main":

```
const x = document.getElementById("main");  
const y = x.getElementsByTagName("p");
```

JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

Finding HTML elements by class name:

- If you want to find all HTML elements with the same class name, use `getElementsByClassName( )`.
- This example returns a list of all elements with `class="intro"`.

```
const x = document.getElementsByClassName("intro");
```

JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

Finding HTML elements by CSS selectors:

- If you want to find all HTML elements that match a specified CSS selector (id, class names, types, attributes, values of attributes, etc), use the `querySelectorAll()` method.
- This example returns a list of all `<p>` elements with `class="intro"`.

```
const x = document.querySelectorAll("p.intro");
```

JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

Finding HTML elements by HTML object collections:

- This example finds the form element with `id="frm1"`, in the forms collection, and displays all element values:

```
const x = document.forms["frm1"];
let text = "";
for (let i = 0; i < x.length; i++) {
    text += x.elements[i].value + "<br>";
}
document.getElementById("demo").innerHTML = text;
```

JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

- HTML DOM allows JavaScript to react to HTML events.
- A JavaScript can be executed when an event occurs, like when a user clicks on an HTML element.
- To execute code when a user clicks on an element, add JavaScript code to an HTML event attribute:

`onclick=JavaScript`

- Examples of HTML events:
 - When a user clicks the mouse
 - When a web page has loaded
 - When an image has been loaded
 - When the mouse moves over an element
 - When an input field is changed
 - When an HTML form is submitted
 - When a user strokes a key

JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

- In this example, the content of the <h1> element is changed when a user clicks on it:

```
<html><body>
```

```
<h1 onclick="this.innerHTML = 'Ooops!'">Click on this  
text!</h1>
```

```
</body></html>
```

JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

- In this example, a function is called from the event handler:

```
<html><body>
```

```
<h1 onclick="changeText(this)">Click on this text!</h1>
```

```
<script>
```

```
function changeText(id) {  
    id.innerHTML = "Ooops!";  
}
```

```
</script>
```

```
</body></html>
```

JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

- To assign events to HTML elements you can use event attributes.

```
<button onclick="displayDate()">Try it</button>
```

- In the example above, a function named `displayDate` will be executed when the button is clicked.

JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

- The HTML DOM allows you to assign events to HTML elements using JavaScript:

```
<script>  
document.getElementById("myBtn").onclick = displayDate;  
</script>
```

- In the example above, a function named `displayDate` is assigned to an HTML element with the `id="myBtn"`.
- The function will be executed when the button is clicked.

JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

- The `onload` and `onunload` events are triggered when the user enters or leaves the page.
- The `onload` event can be used to check the visitor's browser type and browser version, and load the proper version of the web page based on the information.
- The `onload` and `onunload` events can be used to deal with cookies.

```
<body onload="checkCookies()">
```

JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

- The `oninput` event is often to some action while the user input data.
- Below is an example of how to use the `oninput` to change the content of an input field.
`<input type="text" id="fname" oninput="upperCase()">`
- The `onchange` event is often used in combination with validation of input fields.
- Below is an example of how to use the `onchange`.The `upperCase( )` function will be called when a user changes the content of an input field.

`<input type="text" id="fname" onchange="upperCase()">`

JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

- The onmouseover and onmouseout events can be used to trigger a function when the user mouses over, or out of, an HTML element.

```
<html><body>
```

```
<div onmouseover="mOver(this)" onmouseout="mOut(this)" style="background-color:#D94A38;width:120px;height:20px;padding:40px;">Mouse Over Me</div>
```

```
<script>
```

```
function mOver(obj) {
```

```
    obj.innerHTML = "Thank You"
```

```
}
```

```
function mOut(obj) {
```

```
    obj.innerHTML = "Mouse Over Me"
```

```
}
```

```
</script>
```

```
</body></html>
```

JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

- The onmousedown, onmouseup, and onclick events are all parts of a mouse-click. First when a mouse-button is clicked, the onmousedown event is triggered, then, when the mouse-button is released, the onmouseup event is triggered, finally, when the mouse-click is completed, the onclick event is triggered.

```
<html><body>
```

```
<div onmousedown="mDown(this)" onmouseup="mUp(this)" style="background-color:#D94A38;width:90px;height:20px;padding:40px;">Click Me</div>
```

```
<script>
```

```
function mDown(obj) {
```

```
    obj.style.backgroundColor = "#1ec5e5";
```

```
    obj.innerHTML = "Release Me";
```

```
}
```

```
function mUp(obj) {
```

```
    obj.style.backgroundColor="#D94A38";
```

```
    obj.innerHTML="Thank You";
```

```
}
```

```
</script>
```

```
</body></html>
```

JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

- The `addEventListener()` method attaches an event handler to the specified element.
- The `addEventListener()` method attaches an event handler to an element without overwriting existing event handlers.
- You can add many event handlers to one element.
- You can add many event handlers of the same type to one element, i.e two "click" events.
- You can add event listeners to any DOM object not only HTML elements. i.e the window object.
- The `addEventListener()` method makes it easier to control how the event reacts to bubbling.
- When using the `addEventListener()` method, the JavaScript is separated from the HTML markup, for better readability and allows you to add event listeners even when you do not control the HTML markup.
- You can easily remove an event listener by using the `removeEventListener()` method.

JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

- `element.addEventListener(event, function, useCapture);`
- The first parameter is the type of the event (like "click" or "mousedown" or any other HTML DOM Event.)
- The second parameter is the function we want to call when the event occurs.
- The third parameter is a boolean value specifying whether to use event bubbling or event capturing. This parameter is optional.

JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

- Alert "Hello World!" when the user clicks on an element:

```
element.addEventListener("click", function(){  
    alert("Hello World!"); });
```

- You can also refer to an external "named" function:

```
element.addEventListener("click", myFunction);  
function myFunction() {  
    alert ("Hello World!");  
}
```


JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

- The `addEventListener()` method allows you to add many events to the same element, without overwriting existing events:

```
element.addEventListener("click", myFunction);  
element.addEventListener("click", mySecondFunction);
```

- You can add events of different types to the same element:

```
element.addEventListener("mouseover", myFunction);  
element.addEventListener("click", mySecondFunction);  
element.addEventListener("mouseout", myThirdFunction);
```

JAVASCRIPT HTML DOM (DOCUMENT OBJECT MODEL)

- The `removeEventListener()` method removes event handlers that have been attached with the `addEventListener()` method:
`element.removeEventListener("mousemove", myFunction);`